CODE ANALYSIS
I used ChatGPT and Claude for the generated code, I accessed them both through Google.

Correctness
Both programs execute and produce a suitable output, with only a couple differences in terms of correctness. Both programs will accept a puzzle larger than 9x9 but ChatGPT will read it as a 9x9 puzzle, whereas Claude would read it as its size. There was nothing in the requirements about what the program should do in this situation so I think either approach is appropriate. ChatGPT will accept only "_" as a blank whereas Claude would also accept "." and "0" as blanks. The only noticeable edge that one code has on another when it comes to correctness is that if a puzzle were given that had two of the same number in a row or column, ChatGPT could still produce a solution despite there not being any, whereas Claude would not.

Execution Time
Both codes have the exact same algorithm, checking left to right, top to bottom, and they have the same digit order, meaning that any other differences in execution time would likely be negligible, that does not mean there isn't a difference, however. ChatGPT makes a recursive call when it comes across a position with a number in it already while Claude just uses a while loop to skip it which takes up a tiny amount of extra time for ChatGPT.

Space Complexity
ChatGPT does actually have a noticeable edge in space complexity as Claude stores all the solutions where ChatGPT just prints them so if there are very many solutions then Claude's code will take up quite a bit more space then ChatGPT's.

Maintainability
It's become a trend that Claude's code is so much more maintainable than ChatGPT's and this instance is no different. ChatGPT does actually have comments this time but not many and they aren't very descriptive, especially compared to Claude's which has a comment every few lines.

My Work
I decided to pick Claude's code to work with as it is far easier to work with because of how many comments it has. The biggest improvement I saw that could be done was making its space complexity better as that was this code's biggest flaw. Before, it stored all the solutions making the space it took up larger for every solution it had, this can be fixed by not storing the solutions at all and instead, just printing them as soon as they were found. There are about 6.67 sextillion solutions to an empty sudoku board, if the program were given one, it would be running for a very long time, so I thought it would be appropriate to add a cap of 1000, but this could be changed to be higher or lower, depending on what one would like. I also changed the indentation of the classes because I didn't like how it was formatted and added comments to every line.

Prompt:

Create a C++ code using objects, not functions that solves a partially completed 9x9 sudoku puzzle with 3x3 blocks, or, if no solution exists, prove that. To do this, implement a brute-force search that exhaustively tries possibilities until it finds a solution. Use a depth first search with backtracking. For each empty cell it makes an attempt to see how far it can get with it. If it fails to find a solution, it backs up until it finds something else to try, and so on. If a solution exists, it is guaranteed to find it, because it exhaustively tries all possibilities.There are 5 given files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square. Write a recursive object-oriented program to find the solution to the five puzzle files. The output for each solution should include: The puzzle file name, the puzzle from the file printed out, and the solution for the puzzle printed out, or if there is no solution, “no solution found,” printed out. If there is more than one solution to the puzzle, they should all be printed out.

Claude Code

```
// sudoku.cpp
// Object-oriented, recursive, brute-force (depth-first with backtracking)
// Sudoku solver. Finds ALL solutions of each puzzle file, or reports that
// none exist.
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:    ./sudoku            (expects puzzle1.txt ... puzzle5.txt in the
cwd)
//         ./sudoku a.txt b.txt   (optionally name your own files)

#include <array>
#include <cctype>
#include <fstream>
#include <iostream>
#include <iterator>
#include <string>
#include <vector>

//
---------------------------------------------------------------------------
-
// SudokuBoard: owns the 9x9 grid and knows the rules of placement.
//
---------------------------------------------------------------------------
-
class SudokuBoard {
public:
    static const int N = 9;      // board is N x N
    static const int B = 3;      // block is B x B
    static const int EMPTY = 0;  // internal marker for a blank square

    SudokuBoard() { grid_.fill(EMPTY); }

    // Read a puzzle from a file. Digits 1-9 are givens; '_' is a blank.
    // (Whitespace is ignored; '.' and '0' are also accepted as blanks.)
    // Returns false if the file can't be opened or doesn't hold 81 cells.
    bool load(const std::string& filename) {
        std::ifstream in(filename);
```

```
        if (!in) return false;
        std::string text((std::istreambuf_iterator<char>(in)),
                         std::istreambuf_iterator<char>());
        int count = 0;
        for (char c : text) {
            if (count >= N * N) {
                if (std::isdigit(static_cast<unsigned char>(c)) || c ==
'_')
                    return false;  // too many cells
                continue;
            }
            if (c >= '1' && c <= '9')            grid_[count++] = c - '0';
            else if (c == '_' || c == '.' || c == '0') grid_[count++] =
EMPTY;
            // anything else (spaces, newlines, '\r') is skipped
        }
        return count == N * N;
    }

    int  get(int row, int col) const { return grid_[row * N + col]; }
    void set(int row, int col, int v) { grid_[row * N + col] = v; }
    bool isEmpty(int row, int col) const { return get(row, col) == EMPTY;
}

    // Can digit v legally go at (row, col)? Checks row, column and 3x3
block.
    bool canPlace(int row, int col, int v) const {
        for (int i = 0; i < N; ++i) {
            if (get(row, i) == v) return false;
            if (get(i, col) == v) return false;
        }
        int br = (row / B) * B, bc = (col / B) * B;
        for (int r = br; r < br + B; ++r)
            for (int c = bc; c < bc + B; ++c)
                if (get(r, c) == v) return false;
        return true;
    }

    // Are the givens consistent with each other? (No digit repeated in
any
```

```
    // row, column or block.) If not, the puzzle is trivially unsolvable.
    bool givensAreConsistent() const {
        SudokuBoard tmp = *this;
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c) {
                int v = tmp.get(r, c);
                if (v == EMPTY) continue;
                tmp.set(r, c, EMPTY);          // temporarily lift the
digit
                bool ok = tmp.canPlace(r, c, v);
                tmp.set(r, c, v);
                if (!ok) return false;
            }
        return true;
    }

    // Pretty-print with block separators; blanks shown as '_'.
    void print(std::ostream& os) const {
        for (int r = 0; r < N; ++r) {
            if (r > 0 && r % B == 0) os << "------+-------+------\n";
            for (int c = 0; c < N; ++c) {
                if (c > 0 && c % B == 0) os << "| ";
                os << (isEmpty(r, c) ? '_' : static_cast<char>('0' +
get(r, c)));
                os << (c < N - 1 ? " " : "");
            }
            os << '\n';
        }
    }

private:
    std::array<int, N * N> grid_;
};

//
----------------------------------------------------------------------------
-
// SudokuSolver: exhaustive depth-first search with backtracking.
// Works on its own copy of the board and collects every solution found.
```

```
//
------------------------------------------------------------------------
-
class SudokuSolver {
public:
    explicit SudokuSolver(const SudokuBoard& puzzle) : board_(puzzle) {}

    // Run the search; returns the number of solutions found.
    std::size_t solveAll() {
        solutions_.clear();
        if (!board_.givensAreConsistent()) return 0;  // contradictory
givens
        search(0);
        return solutions_.size();
    }

    const std::vector<SudokuBoard>& solutions() const { return solutions_;
}

private:
    SudokuBoard board_;
    std::vector<SudokuBoard> solutions_;

    // Recursive step: handle the cell at linear index `pos` (0..80).
    void search(int pos) {
        const int total = SudokuBoard::N * SudokuBoard::N;

        // Skip over cells that are already filled in.
        while (pos < total && !board_.isEmpty(pos / SudokuBoard::N,
                                              pos % SudokuBoard::N))
            ++pos;

        // Every cell is filled: record this solution, then return so the
        // caller backs up and keeps looking for others (exhaustive
search).
        if (pos == total) {
            solutions_.push_back(board_);
            return;
        }
```

```
        int row = pos / SudokuBoard::N;
        int col = pos % SudokuBoard::N;

        for (int v = 1; v <= SudokuBoard::N; ++v) {
            if (board_.canPlace(row, col, v)) {
                board_.set(row, col, v);        // try it
                search(pos + 1);                // go deeper
                board_.set(row, col, SudokuBoard::EMPTY);  // backtrack
            }
        }
        // Falling out of the loop = dead end here; the caller will back
up.
    }
};

//
-------------------------------------------------------------------------
-
// PuzzleRunner: loads one file, solves it, and reports the results.
//
-------------------------------------------------------------------------
-
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string& filename) :
filename_(filename) {}

    void run(std::ostream& os) {
        os << "========================================\n";
        os << "Puzzle file: " << filename_ << "\n\n";

        SudokuBoard puzzle;
        if (!puzzle.load(filename_)) {
            os << "Error: could not read a valid 9x9 puzzle from this
file.\n\n";
            return;
        }

        os << "Puzzle:\n";
        puzzle.print(os);
```

```
        SudokuSolver solver(puzzle);
        std::size_t n = solver.solveAll();

        os << '\n';
        if (n == 0) {
            os << "no solution found\n\n";
            return;
        }
        os << n << (n == 1 ? " solution" : " solutions") << " found:\n";
        int i = 1;
        for (const SudokuBoard& s : solver.solutions()) {
            os << "\nSolution " << i++ << ":\n";
            s.print(os);
        }
        os << '\n';
    }

private:
    std::string filename_;
};

//
------------------------------------------------------------------------
-
int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    } else {
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".txt");
    }

    for (const std::string& f : files) {
        PuzzleRunner runner(f);
        runner.run(std::cout);
    }
    return 0;
}
```

ChatGPT Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku {
private:
    char grid[9][9];
    char original[9][9];
    int solutionCount;

    // Check whether a digit can be placed in a cell.
    bool isValid(int row, int col, char digit) {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == digit)
                return false;

            if (grid[i][col] == digit)
                return false;
        }

        // Find the top-left corner of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check the 3x3 block.
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == digit)
                    return false;
            }
        }

        return true;
    }

    // Find and print every solution using recursion.
```

```
    void search(int position) {
        // All 81 cells have been processed.
        if (position == 81) {
            solutionCount++;
            cout << "\nSolution " << solutionCount << ":\n";
            printGrid(grid);
            return;
        }

        int row = position / 9;
        int col = position % 9;

        // Skip cells that already contain a given digit.
        if (grid[row][col] != '_') {
            search(position + 1);
            return;
        }

        // Try every possible digit in the empty cell.
        for (char digit = '1'; digit <= '9'; digit++) {
            if (isValid(row, col, digit)) {
                grid[row][col] = digit;

                // Recursively solve the remaining cells.
                search(position + 1);

                // Backtrack: undo the attempted digit.
                grid[row][col] = '_';
            }
        }
    }

    // Print a Sudoku grid.
    void printGrid(char board[9][9]) {
        for (int row = 0; row < 9; row++) {
            if (row % 3 == 0 && row != 0)
                cout << "------+-------+------\n";

            for (int col = 0; col < 9; col++) {
                if (col % 3 == 0 && col != 0)
```

```
                    cout << "| ";

                cout << board[row][col] << ' ';
            }

            cout << '\n';
        }
    }

public:
    // Read a puzzle from a file.
    bool loadPuzzle(const string& filename) {
        ifstream input(filename);

        if (!input) {
            cout << "Could not open " << filename << '\n';
            return false;
        }

        int count = 0;
        char value;

        while (input >> value && count < 81) {
            if (value >= '1' && value <= '9') {
                grid[count / 9][count % 9] = value;
            } else if (value == '_') {
                grid[count / 9][count % 9] = '_';
            } else {
                // Ignore separators or other non-cell characters.
                continue;
            }

            count++;
        }

        if (count != 81) {
            cout << "Invalid puzzle file: " << filename << '\n';
            return false;
        }
```

```
        // Save the original puzzle for printing.
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                original[row][col] = grid[row][col];
            }
        }

        return true;
    }

    // Solve the puzzle and display the results.
    void solve(const string& filename) {
        solutionCount = 0;

        cout << "\n================================\n";
        cout << "Puzzle file: " << filename << '\n';

        cout << "\nOriginal puzzle:\n";
        printGrid(original);

        search(0);

        if (solutionCount == 0) {
            cout << "\nno solution found\n";
        } else {
            cout << "\nTotal solutions found: "
                 << solutionCount << '\n';
        }
    }
};

int main() {
    string filenames[] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
```

```
    for (int i = 0; i < 5; i++) {
        Sudoku puzzle;

        if (puzzle.loadPuzzle(filenames[i])) {
            puzzle.solve(filenames[i]);
        }
    }

    return 0;
}
```